Manual de Implementación: Heladería Inteligente con IA Local

Proyecto Raspberry Pi, Python y LLMs Offline

HAAS - UNIVALLE

18 de febrero de 2026

Índice general

1. Introducción y Arquitectura **2**
1.1. Objetivo del Proyecto 2
1.2. Stack Tecnológico (Gratuito) 2

2. Estructura del Proyecto **3**

3. Implementación del Código Base **4**
3.1. Paso 1: Base de Datos 4
3.2. Paso 2: API (Backend) 4
3.3. Paso 3: Dashboard (Frontend) 5

4. Configuración del Servidor (Raspberry Pi) **6**
4.1. Instalación de Dependencias 6
4.2. Configuración de Inicio Automático (Systemd) 6
4.2.1. Servicio API 6
4.2.2. Servicio App 6

5. Inteligencia Artificial Local y Voz **8**
5.1. Instalación de Ollama y Piper 8
5.2. Agente Final con Voz 8

6. Conclusión **10**

Capítulo 1

Introducción y Arquitectura

1.1. Objetivo del Proyecto

Crear un sistema de administración para una heladería que funcione totalmente **offline** en una Raspberry Pi. El sistema incluye una base de datos, una API, un panel de control y un Agente de Inteligencia Artificial capaz de hablar y gestionar ventas.

1.2. Stack Tecnológico (Gratuito)

- **Base de Datos:** SQLite (Archivo local, sin servidor).
- **Backend:** FastAPI (Python).
- **Frontend:** Streamlit (Dashboard web).
- **Inteligencia Artificial:** Ollama (Modelo Llama 3.2).
- **Voz (TTS):** Piper (Neural TTS optimizado para Raspberry Pi).
- **Hardware:** Raspberry Pi 4 (8GB) o Raspberry Pi 5.

Capítulo 2

Estructura del Proyecto

Se debe crear la siguiente estructura de carpetas en el directorio `/home/pi/heladeria`:

```
heladeria/
|-- db/
|   |-- database.py     # Script de inicialización
|   |-- heladeria.db    # Archivo generado automáticamente
|-- api/
|   |-- main.py         # Servidor FastAPI
|-- app/
|   |-- dashboard.py    # Interfaz Streamlit
|-- mcp/
|   |-- server.py       # Servidor MCP (Opcional)
|-- local_agent_voz.py  # Script principal de la IA
|-- venv/               # Entorno virtual de Python
```

Capítulo 3

Implementación del Código Base

3.1. Paso 1: Base de Datos

Archivo: `db/database.py`

```python
import sqlite3

def init_db():
    conn = sqlite3.connect('heladeria.db')
    c = conn.cursor()
    # Tabla de Sabores
    c.execute'''
        CREATE TABLE IF NOT EXISTS sabores (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nombre TEXT NOT NULL,
            precio REAL NOT NULL,
            stock INTEGER NOT NULL
        )
    '''')
    # Datos iniciales (Seed)
    c.execute("INSERT OR IGNORE INTO sabores (id, nombre, precio, stock) VALUES (1, 'Vainilla', 3.50, 50)")
    c.execute("INSERT OR IGNORE INTO sabores (id, nombre, precio, stock) VALUES (2, 'Chocolate', 4.00, 40)")
    conn.commit()
    conn.close()
    print("Base de datos creada exitosamente.")

if __name__ == "__main__":
    init_db()
```

3.2. Paso 2: API (Backend)

Archivo: `api/main.py`

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import sqlite3

app = FastAPI()

class Venta(BaseModel):
    sabor_id: int
    cantidad: int

def get_db_connection():
    conn = sqlite3.connect('../db/heladeria.db')
```

```python
    conn.row_factory = sqlite3.Row
    return conn

@app.get("/sabores")
def leer_inventario():
    conn = get_db_connection()
    sabores = conn.execute('SELECT * FROM sabores').fetchall()
    conn.close()
    return {"inventario": sabores}

@app.post("/vender")
def vender_helado(venta: Venta):
    conn = get_db_connection()
    cursor = conn.cursor()

    # Verificar stock
    sabor = cursor.execute('SELECT stock FROM sabores WHERE id = ?', (venta.sabor_id,)).fetchone()
    if not sabor or sabor['stock'] < venta.cantidad:
        raise HTTPException(status_code=400, detail="Stock insuficiente")

    # Restar stock
    cursor.execute('UPDATE sabores SET stock = stock - ? WHERE id = ?', (venta.cantidad, venta.sabor_id))
    conn.commit()
    conn.close()
    return {"mensaje": "Venta realizada", "cantidad_vendida": venta.cantidad}
```

3.3. Paso 3: Dashboard (Frontend)

Archivo: app/dashboard.py

```python
import streamlit as st
import requests
import pandas as pd

API_URL = "http://127.0.0.1:8000"

st.title("Administracion Heladeria")

# 1. Mostrar Inventario
st.header("Inventario Actual")
if st.button("Actualizar Tabla"):
    response = requests.get(f"{API_URL}/sabores")
    if response.status_code == 200:
        data = response.json()["inventario"]
        df = pd.DataFrame(data)
        st.dataframe(df)

# 2. Registrar Venta Manual
st.header("Registrar Venta")
id_sabor = st.number_input("ID del Sabor", min_value=1, step=1)
cantidad = st.number_input("Cantidad de bolas", min_value=1, step=1)

if st.button("Vender"):
    payload = {"sabor_id": id_sabor, "cantidad": cantidad}
    res = requests.post(f"{API_URL}/vender", json=payload)
    if res.status_code == 200:
        st.success("Venta registrada!")
    else:
        st.error(f"Error: {res.json().get('detail')}")
```

Capítulo 4

Configuración del Servidor (Raspberry Pi)

4.1. Instalación de Dependencias

Ejecutar en la terminal de la Raspberry Pi:

```
# Actualizar sistema
sudo apt update && sudo apt upgrade -y
sudo apt install python3-venv git alsa-utils -y

# Crear entorno virtual
python3 -m venv venv
source venv/bin/activate

# Instalar librerías Python
pip install fastapi uvicorn streamlit requests mcp ollama
```

4.2. Configuración de Inicio Automático (Systemd)

4.2.1. Servicio API

Crear archivo: `/etc/systemd/system/heladeria-api.service`

```
[Unit]
Description=API Heladeria
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/heladeria/api
Environment="PATH=/home/pi/heladeria/venv/bin"
ExecStart=/home/pi/heladeria/venv/bin/uvicorn main:app --host 0.0.0.0 --port
    8000
Restart=always

[Install]
WantedBy=multi-user.target
```

4.2.2. Servicio App

Crear archivo: `/etc/systemd/system/heladeria-app.service`

```
[Unit]
Description=App Heladeria Streamlit
```

```
After=heladeria-api.service

[Service]
User=pi
WorkingDirectory=/home/pi/heladeria/app
Environment="PATH=/home/pi/heladeria/venv/bin"
ExecStart=/home/pi/heladeria/venv/bin/streamlit run dashboard.py --server.port
    8501 --server.address 0.0.0.0
Restart=always

[Install]
WantedBy=multi-user.target
```

Activar servicios:

```
sudo systemctl enable heladeria-api heladeria-app
sudo systemctl start heladeria-api heladeria-app
```

Capítulo 5

Inteligencia Artificial Local y Voz

5.1. Instalación de Ollama y Piper

```
# 1. Instalar Ollama
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl start ollama

# 2. Descargar Modelo Llama 3.2
ollama pull llama3.2

# 3. Instalar Piper (Voz)
cd ~/heladeria
wget https://github.com/rhasspy/piper/releases/download/v1.2.0/
    piper_linux_aarch64.tar.gz
tar -xvf piper_linux_aarch64.tar.gz

# 4. Descargar voz en español
wget -O voz_es.onnx https://huggingface.co/rhasspy/piper-voices/resolve/main/es/
    es_MX/aldona/medium/es_MX-aldona-medium.onnx?download=true
wget -O voz_es.onnx.json https://huggingface.co/rhasspy/piper-voices/resolve/
    main/es/es_MX/aldona/medium/es_MX-aldona-medium.onnx.json?download=true
```

5.2. Agente Final con Voz

Archivo: `local_agent_voz.py`

Este script integra la IA (Ollama), el sistema de voz (Piper) y el API de la heladería.

```
import ollama
import requests
import json
import subprocess

API_URL = "http://127.0.0.1:8000"
MODELO = "llama3.2"
PIPER_BIN = "./piper/piper"
VOZ_MODELO = "voz_es.onnx"

def hablar(texto):
    print(f"Diciendo: {texto}")
    try:
        clean_text = texto.replace("*", "").replace("#", "")
        cmd = f'echo "{clean_text}" | {PIPER_BIN} --model {VOZ_MODELO} --
    output_file salida.wav'
        subprocess.run(cmd, shell=True, check=True)
        subprocess.run("aplay -q salida.wav", shell=True)
    except Exception as e:
```

```
        print(f"Error audio: {e}")

# ... (El resto de funciones consultar_sabores y registrar_venta son iguales a la Clase 4) ...
# ... (El bucle principal llama a hablar() despues de recibir respuesta de Ollama) ...
```

Capítulo 6

Conclusión

Con este manual, has transformado una Raspberry Pi en un servidor de borde (Edge Server) completo, capaz de gestionar inventario real y comunicarse verbalmente con los clientes sin depender de internet.